

Esame OS Internals (Cabodi)

Data: 07/07/2023

Nota Generale: In tutte le domande, ogni volta che il risultato è una selezione, un SI/NO, o un numero, sono necessari una motivazione/spiegazione o passaggi intermedi che portino al risultato.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Sostituzione Pagine e Memoria Virtuale)

Contesto: Si consideri la seguente stringa di riferimenti di memoria, per un determinato processo. Per ogni riferimento (Byte addressing, con indirizzi espressi in esadecimale) viene riportata anche l'operazione di lettura/scrittura (R/W):

R 33FB, R 1B64, W 30D3, W 237E, R 0AC8, W 23D7, R 174A, R 0965, W 32A0, R 1BB0, W 09E5, R 3380, R 2A94, R 11B8.

Si supponga che gli indirizzi fisici e logici siano su 16 bit, la dimensione della pagina sia 2KByte e 70FF sia l'indirizzo massimo utilizzabile dal programma (il limite superiore dello spazio degli indirizzi).

- A) Calcolare la dimensione dello spazio degli indirizzi (address space, espresso come numero di pagine) e la frammentazione interna.
- A) Calcolare la stringa di riferimenti di pagina.
- A) Simulare un algoritmo di sostituzione di pagine di tipo second-chance, con 3 frame disponibili. Rappresentare il resident set (frame fisici contenenti pagine logiche) dopo ogni riferimento in memoria. Indicare esplicitamente, per ogni frame allocato, il numero di pagina e il bit di riferimento (utilizzare la notazione p_r). Indicare anche i page fault. Si supponga che il bit di riferimento di una pagina sia inizializzato a 0 dopo il relativo page fault.

Domanda 2 (HDD, Geometria e Prestazioni)

Contesto: Si consideri un'unità disco rigido (HDD) con tecnologia e geometria standard. Il disco ha: dimensione del settore di 512 Byte, 2048 tracce per faccia, 50 settori per traccia, 5 dischi/piatti a doppia faccia, Tempo medio di seek di 10 msec.

- A) Qual è la capacità di una traccia (quanti Byte può contenere)? Qual è la capacità di ogni faccia? Qual è la capacità del disco? Quanti cilindri ha il disco?
- A) Il metodo di organizzazione di disco CHS (Cylinder-Head-Sector) consente a un blocco del disco di estendersi su più settori di una data traccia (identificata da una coppia cilindro, testina) ma non su tracce diverse. Le seguenti dimensioni dei blocchi sono valide? 256 B, 2048 B, 51200 B (rispondere per ognuna e motivare le risposte).

- A) Se i dischi ruotano a 5400 giri/min (giri al minuto), qual è la latenza rotazionale massima? Qual è la latenza rotazionale media? Si sa che è possibile trasferire un'intera traccia di dati per rotazione, qual è la velocità di trasferimento (espressa in bit/secondo)? Quale sarebbe la velocità di trasferimento se un intero cilindro di dati potesse essere trasferito in una rotazione?
-

Domanda 3 (File System, Allocazione e Archiviazione)

- A) Un'applicazione deve organizzare un file con un'intestazione (header) e più sezioni. L'header include, tra gli altri dati, offset e dimensioni di tutte le sezioni del file.
1. L'applicazione può supporre che l'header, così come ogni sezione inizi all'inizio di un blocco del disco (nel file system)? (Motivare)
 2. Può l'applicazione decidere di organizzare i dati in più file (anziché in un singolo file): un file per l'intestazione e uno per ogni sezione? (Se NO, motivare, se SI, motivare e spiegare come gli offset di sezione nell'intestazione devono essere rappresentati, codificati o modificati).
- A) Un file di testo viene copiato in tre diversi file system, con formati di allocazione file, rispettivamente: allocazione contigua (per multiplo di un blocco di disco), allocazione collegata (senza FAT), e allocazione basata su Inode. La dimensione di un blocco su disco è la stessa in tutti e tre i file system. Possiamo supporre che la frammentazione interna sarà la stessa in tutti i file system? (Rispondere e motivare).
- A) Un file system contiene tre file di testo `a.txt`, `b.txt` e `c.txt`. Un'applicazione di archiviazione/compressione di file (come ad esempio gzip, tar, 7z, rar, ecc.) memorizza il (contenuto dei) tre file di testo all'interno di un singolo file di archivio (ad esempio `abc.zip`).
1. I tre file di testo possono condividere lo stesso blocco del disco, il che significa che il file system li memorizza (almeno parzialmente) all'interno dello stesso blocco del disco? (Motivare)
 2. È possibile, per il contenuto dei tre file di testo, replicati e/o compressi nel file di archivio, condividere (almeno parzialmente) lo stesso blocco disco (in modo da essere memorizzato all'interno dello stesso blocco disco)? (Motivare)

2 SECTION 2: OS161

Domanda 4 (File System Framework in OS161)

Contesto: E' dato un sistema OS161. Si consideri il framework del file system utilizzato in LAB5, parzialmente rappresentato dai seguenti frammenti di codice (da `proc.h` e `file_syscalls.c`):

```
struct proc {
#ifdef OPT_FILE
    /* per-process open file table */
    struct openfile *fileTable[OPEN_MAX];
#endif
};

/* system open file table */
struct openfile {
    struct vnode *vn;
    off_t offset;
    unsigned int countRef;
};

struct openfile systemFileTable[SYSTEM_OPEN_MAX];
```

Rispondere alle seguenti domande:

- B) Perché `systemFileTable` è un array di `struct openfile`, mentre `fileTable` è un array di puntatori a `struct openfile`?
 - B) Il campo `countRef` è ridondante, considerando che il `vnode` puntato dal campo `vn` ha (internamente) il suo contatore di riferimenti?
 - B) Nel caso in cui sia necessario supportare operazioni di lock dei file, cosa andrebbe scelto tra uno spinlock e un lock e dove dovrebbe essere posizionato (o posizionati, se multipli): in `systemFileTable`, in `fileTable` o altrove?
 - B) Qual è il possibile ruolo delle funzioni `copyin` e `copyout` nell'implementazione delle chiamate di sistema `sys_read` e `sys_write`?
-

Domanda 5 (Sincronizzazione e Spinlock)

Contesto: Si considerino, in OS161, le due possibili implementazioni (parziali) di `spinlock_acquire` mostrate di seguito:

```
void spinlock_acquire(struct spinlock *splk) {
    // ...
    while (1) {
        if (spinlock_data_get(&splk->splk_lock) != 0) {
            continue;
        }
        if (spinlock_data_testandset(&splk->splk_lock) != 0) {
            continue;
        }
    }
}
```

```

        break;
    }
    // ...
}

void spinlock_acquire2(struct spinlock *splk) {
    // ...
    while (1) {
        while (spinlock_data_get(&splk->splk_lock) != 0);
        if (spinlock_data_testandset(&splk->splk_lock) == 0) {
            continue;
        }
    }
    // ...
}

```

Rispondere alle seguenti domande:

- B)** Perché `spinlock_acquire` utilizza sia `spinlock_data_get` che `spinlock_data_testandset`, invece di chiamarne solo una?
- B)** `spinlock_acquire2` equivale a `spinlock_acquire`? Se sì, motivarlo, se no, motivarlo e, se possibile, modificarla per renderla equivalente.